Forensic Audit Logging

Workstream Update



Some background...

What is a Forensic Audit?

A forensic audit is an in-depth examination of records designed to uncover evidence for use in a court of law.

What does Forensic Audit mean for Mojaloop?



- Mojaloop is used to build systems which involve the accounting and movement of money and personal data between institutions
- Money and personal data are favourite targets for criminal enterprise

- Mojaloop schemes are potential vectors for financial crime, either as a means of moving illegally obtained funds quickly, between transaction accounts, or as a source of money to be illegally obtained e.g. theft by insiders.
- Mojaloop schemes are potentially stores of a vast volume of valuable personal data

Mojaloop schemes therefore need to support investigations of crime in as reliable and straightforward a manner as possible.

The Mojaloop Forensic Audit Log



To support such investigations, the Mojaloop platform must include a highly accurate record keeping system…

- To support investigations into activities that have occurred within the hub system boundary.

- To provide **verifiable** data, that would stand up to adversarial scrutiny in a court of law, in support of ascertaining the chain of events surrounding events of interest, with a view to identifying the actors and the actions they undertook which resulted in a particular outcome. E.g. "*What caused that ledger entry to be recorded?*"

Forensic Audit Log Functional Requirements

- The system must record rich data around all significant user and system activities
  - E.g. all operator actions and requests relating to money movements and access to personal data

- Records must be resistant to tampering
  - Tamper-evident rather than tamper-proof
  - Must stand up in court - mathematical proof

- The system must record events in as near real-time as possible
  - Ability to detect adverse events rapidly

- Audit records must be searchable to support enquiries

etc…

Forensic Audit Log Non-Functional Requirements



- The system must support the recording of a large volume of events in an efficient manner
  - Low processing cost, low storage cost, scalability

- Records must be durable for a configurable period
  - Multi-year regulatory storage requirements

- Processing and storing audit trail records must have minimal impact on overall system performance
  - < 5%

- Failure to durably record an action in the forensic audit log must prevent the action from succeeding
  - No unaudited events can be allowed to happen in the system

etc…

Hang on! Didn’t we do this before?

In the early days of Mojaloop (9 years ago), a forensic logging sidecar and centralised key management service service were developed.

This solution quickly fell out of use due to lack of demand at the time and was archived in 2021.

The sidecar-and-KMS solution paired a lightweight sidecar deployed alongside each service which locally logs events, signs individual rows with a symmetric key and groups them into batches signed with an asymmetric key. The central KMS registers each sidecar, issues and tracks its signing keys, and can audit any sidecar's batches on demand.



Hang on! Didn’t we do this before?



Q: Why didn’t we just resurrect the existing code?

A: Recent demand gave us an opportunity to reexamine requirements for this feature and judge the relative merits of the existing solution against alternatives.

Things we didn’t like about the existing code:

- DBMS storage is expensive compared to object storage for the volumes of data and retention periods Mojaloop is currently expecting to handle
- Home-grown crypto glue - creating and maintaining security related code is high risk compared to using off-the-shelf tried and tested alternatives
- Inferior tamper resistance characteristics compared to more recently considered alternatives

How do we meet the requirements?

MojaCom

Question: Are there any off-the-shelf solutions that meet all our requirements?

Short answer: no.

Long answer: Although there are no complete solutions that meet our functional and non-functional requirements, and that are compatible with our open-source licensing, there are some “building blocks” we can use to construct a good solution e.g. Merkle chain DBMS, object storage, PKI tools etc…

How do we meet the requirements?



Hardest problems:

1. Large volume and rate of events needing to be recorded in a cost effective manner. Some projections suggest ~6000 events per second (1000TPS sustained) @ ~4kb per event ~2TB per day
   - Disk storage is not cheap (x2-3 replicated)

1. Hardening against tampering
   - Specialised write-once-read-many (WORM) data storage solutions are not cheap

1. Preventing an auditable event from succeeding if audit write fails means using blocking calls all the way through the write path; queues, streams just won't do.
   - Performance and reliability must be flawless.

How do we meet the requirements?



Problem: How to store a large volume of events, quickly and in an efficient, cost effective manner.

Solution: Object storage e.g. S3 compatible, scalable, distributed storage.

Issue: S3 options are not natively tamper evident

How do we meet the requirements?

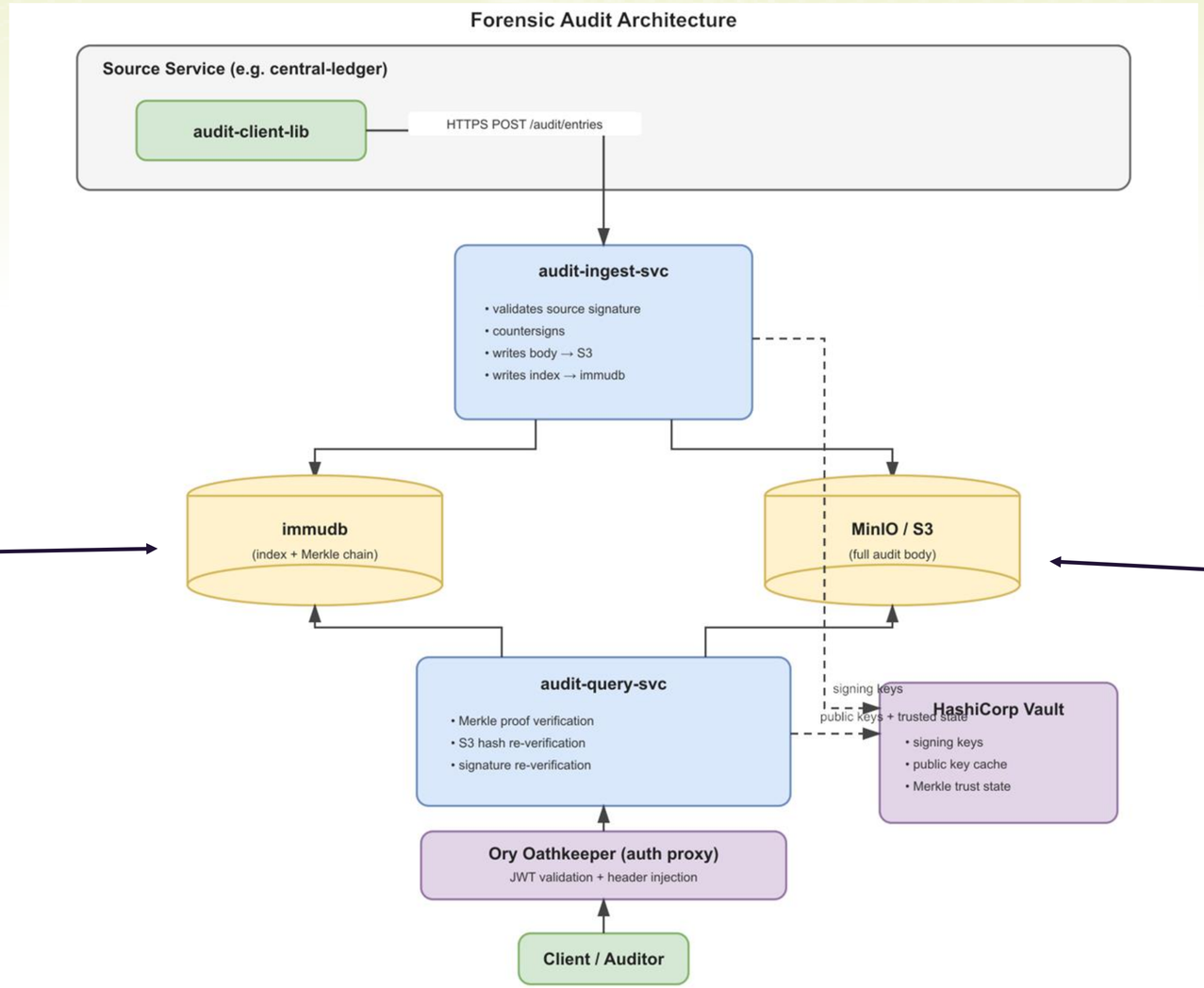


Problem: How do we utilize the cheap, scalable nature of object storage while maintaining strong tamper resistance?

Solution: Store forensic audit event data in cheap, scalable object storage but keep a tamper evident “index” in a specialised Merkle chain data store (immudb)

*Note: A **Merkle chain** links each event by hashing its contents together with the previous event's hash, so every hash commits to the entire history up to that point. Because changing any earlier event changes all subsequent hashes, publishing or securely storing the latest hash makes any later tampering with the sequence immediately detectable.*

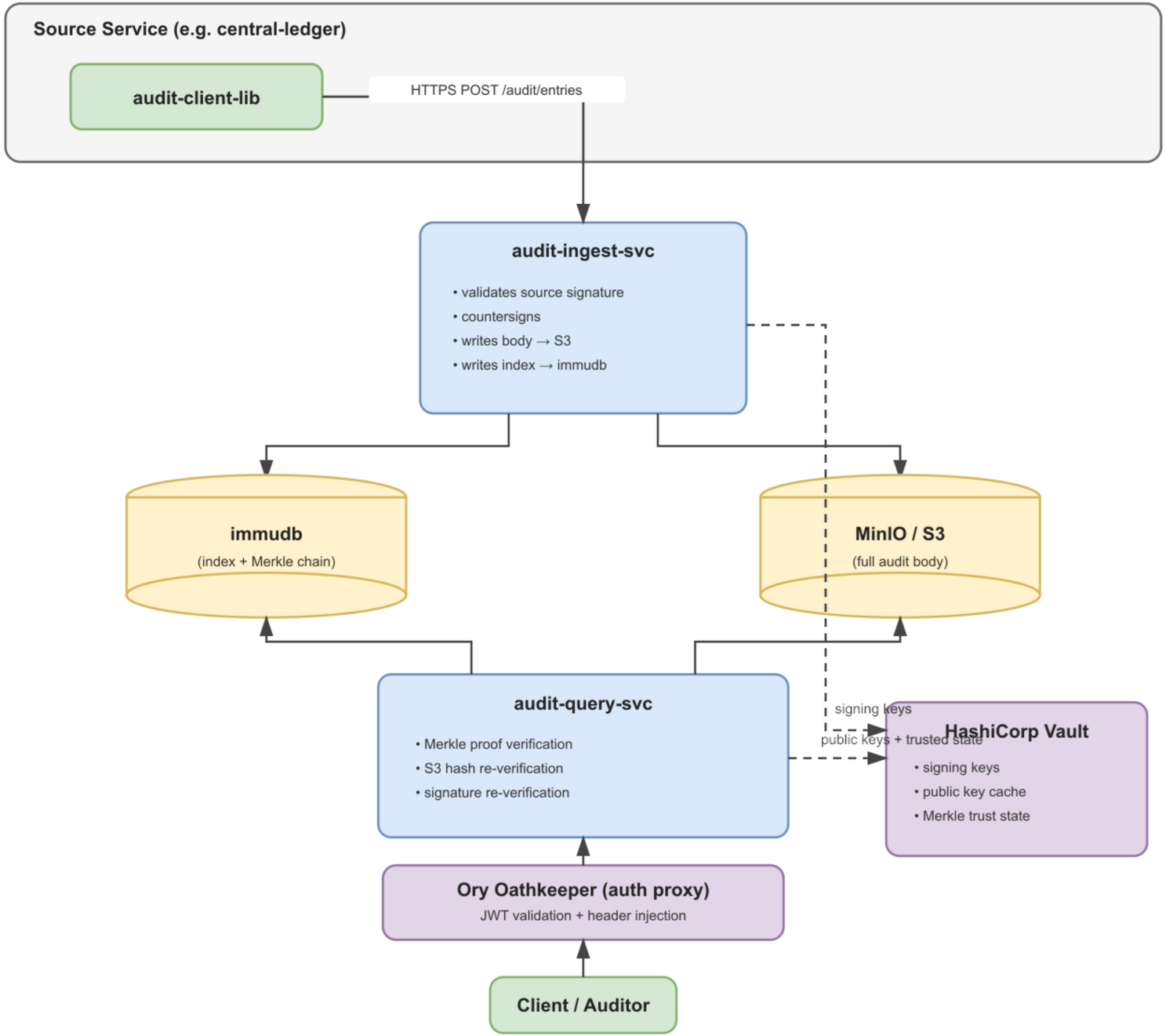
How do we meet the requirements?



immudb stores a **merkle chain index** of all audit event records (timestamp, actors, body hash). Index entries are small so storage requirements are modest

S3 object storage stores the full **“body” of the audit event**

Forensic Audit Architecture



Current Status

- An implementation of this new design has been produced.
  - TypeScript
  - AI assisted coding with Claude Sonnet 4.6, fully human verified.
  - Performance testing underway.
  - Performance test results will validate or invalidate some design choices e.g. blocking vs asynchronous writes, scalability of immudb
- Code reviews underway but more eyes are needed!
- Design review welcomed: https://github.com/mojaloop/forensic-audit/blob/main/docs/design.md

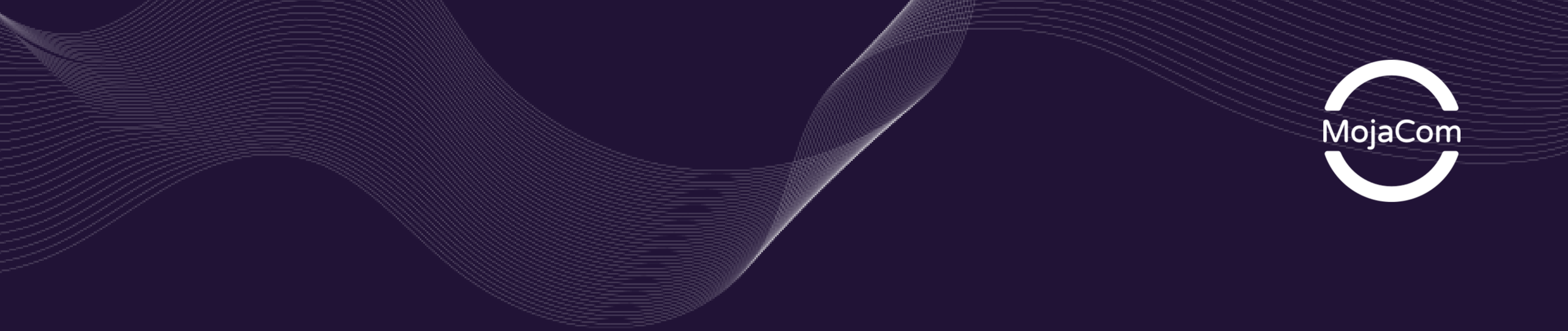
Reach out to participate!

Email: james.bush@mojaloop.io

Mojaloop Slack: #ws-forensic-audit







Thank you for listening!

James Bush <jbush@mojaloop.io>